

# RECIPE SHELTER

Site Web de Recettes de Cuisine Collaboratif

## SYNTHÈSE DES COMPÉTENCES MOBILISÉES

### Blocs de compétences couverts :

Bloc 1 — Intégration Front-End  
Bloc 2 — Développement Back-End  
Bloc 3 — Framework

### Stack technique :

Angular 21 · TypeScript · Node.js / Express · MySQL 8 · JWT · Angular SSR · Bootstrap 5

# 1. Présentation du projet

## 1.1 Contexte et objectif

Recipe Shelter est un site web collaboratif de partage de recettes de cuisine développé dans le cadre de la formation RNCP Développeur Web. Le projet couvre les trois blocs de compétences du référentiel : intégration front-end, développement back-end et développement avec framework.

L'objectif était de créer une application web complète, allant de la maquette à la mise en production, en passant par le développement full-stack, la gestion d'une base de données relationnelle et l'intégration d'un framework JavaScript moderne.

## 1.2 Fonctionnalités développées

- Consultation, recherche avancée et filtrage des recettes publiques (par catégorie, tag, ingrédient, temps de préparation)
- Système de comptes utilisateurs avec inscription, validation d'email, connexion et réinitialisation de mot de passe
- Gestion des recettes : création en brouillon, soumission à modération, publication, archivage
- Commentaires hiérarchiques (1 niveau de réponse) avec notation (1 à 5 étoiles) sur les recettes
- Système de favoris pour les utilisateurs connectés
- Espace d'administration : modération des recettes, des commentaires et gestion des utilisateurs (bannissement)
- Profil public par nom d'utilisateur, espace privé connecté
- Formulaire de contact avec envoi d'email (Nodemailer / SMTP)
- Pages légales : mentions légales, CGU, politique de confidentialité
- Server-Side Rendering (SSR) Angular pour l'optimisation SEO et les performances

# 2. Tableau récapitulatif des technologies mobilisées

| Bloc 1 — Front-End                                 | Bloc 2 — Back-End                  | Bloc 3 — Framework              |
|----------------------------------------------------|------------------------------------|---------------------------------|
| HTML5 sémantique                                   | Node.js + Express 5 + TypeScript   | Angular 21 (framework choisi)   |
| CSS3 (flexbox, grid, variables)                    | Architecture en couches (C/S/R)    | Composants standalone           |
| JavaScript (DOM, fetch, async/await)               | MySQL 8 + requêtes optimisées      | Routing modulaire lazy-loaded   |
| CSS3 pur (flexbox, grid, variables, media queries) | JWT + cookie HttpOnly + bcrypt     | Signals pour la gestion d'état  |
| Responsive (media queries)                         | Validation email + reset password  | Interceptor HTTP pour le JWT    |
| Formulaires & validation HTML5                     | SMTP (Nodemailer)                  | SSR avec Express intégré        |
| Accessibilité WCAG / ARIA                          | Tests unitaires (Node test runner) | Vitest pour les tests unitaires |
| Performance & optimisation                         | Rate limiting, CORS, sécurité      | Lint-staged + Husky pre-commit  |

## 3. Bloc 1 — Intégration Front-End

### 3.1 Description de l'approche

L'intégration front-end repose sur les langages fondamentaux du web : HTML5 pour la structure sémantique des pages, CSS3 pour la mise en forme et la responsivité, et JavaScript pour les interactions côté navigateur. Aucun framework CSS n'a été utilisé : l'ensemble des styles est écrit en CSS pur dans un fichier unique (main.css), avec des variables CSS globales, des grilles auto-adaptatives (auto-fill / minmax) et 4 breakpoints custom.

Chaque page respecte les principes d'accessibilité web (WCAG) : balises sémantiques, rôles ARIA sur les éléments interactifs, labels associés aux champs de formulaire, et contrastes suffisants. Le site s'adapte à tous les formats d'écran via des media queries CSS pur :  $\leq 991\text{px}$  (menu hamburger, barre de recherche pleine largeur),  $\leq 575\text{px}$  (grilles en colonne unique),  $\geq 961\text{px}$  et  $\leq 760\text{px}$  pour la page détail recette.

### 3.2 Compétences mobilisées

| Bloc 1 Intégration Front-End        |                                                                                                                                                                                                                                                                                                                |                                                                                                                                                 |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Compétence                          | Mise en œuvre dans Recipe Shelter                                                                                                                                                                                                                                                                              | Preuve / Livrable                                                                                                                               |
| <b>HTML5 sémantique</b>             | Structure des pages avec les balises sémantiques HTML5 : header, main, nav, section, article, footer. Hiérarchie des titres h1-h6 respectée. Attributs alt sur les images.                                                                                                                                     | <i>Pages HTML statiques : index.html, recipes.html, recipe-detail.html, sign-in.html, sign-up.html</i>                                          |
| <b>CSS3 &amp; mise en forme</b>     | Styles CSS3 : flexbox, grid, variables CSS, transitions. Feuilles de style par composant. Utilisation de Stylelint pour la qualité et la conformité du CSS.                                                                                                                                                    | <i>css/main.css (fichier CSS unique, ~700 lignes)</i>                                                                                           |
| <b>Responsive Design</b>            | CSS pur, sans framework. 4 breakpoints custom ( $\leq 991\text{px}$ , $\leq 575\text{px}$ , $\geq 961\text{px}$ , $\leq 760\text{px}$ ) et grilles CSS auto-adaptatives (repeat(auto-fill, minmax(...))) pour un nombre de colonnes dynamique sans breakpoints codés en dur.                                   | <i>Media queries dans css/main.css et styles inline dans recipe-detail.html</i>                                                                 |
| <b>JavaScript</b>                   | JavaScript pour les interactions côté client : gestion des événements DOM, soumission de formulaires, affichage et masquage dynamique d'éléments, appels API asynchrones (fetch / async-await).                                                                                                                | <i>js/main.js (navigation, favoris), js/search.js (filtres, recherche), js/signin.js (connexion), js/validation.js (validation formulaires)</i> |
| <b>Formulaires &amp; validation</b> | Formulaires HTML5 avec validation : champs requis, formats (email, longueur min/max). Messages d'erreur contextuels affichés en temps réel. Validation côté client avant envoi.                                                                                                                                | <i>Formulaires d'inscription, de connexion, de recette</i>                                                                                      |
| <b>Accessibilité (WCAG)</b>         | Attributs ARIA (aria-label, aria-describedby, role) sur les éléments interactifs. Labels for associés aux inputs. Navigation clavier opérationnelle. Contrastes couleur conformes WCAG AA.                                                                                                                     | <i>Attributs ARIA dans les templates HTML</i>                                                                                                   |
| <b>Performance front-end</b>        | Attributs loading="eager" et fetchpriority="high" sur l'image hero (dessus de la ligne de flottaison) ; loading="lazy" sur les images de cartes recettes. Attribut decoding="async" pour ne pas bloquer le thread principal. Dimensions width/height explicites sur toutes les balises img pour éviter le CLS. | <i>Assets, build de production</i>                                                                                                              |

## 4. Bloc 2 — Développement Back-End

### 4.1 Architecture et choix techniques

Le back-end est développé en Node.js (LTS) avec Express 5 et TypeScript 5.9. Le code est organisé en trois couches clairement séparées : Controllers (traitement des requêtes HTTP et validation des DTO), Services (logique métier et règles de gestion), Repositories (accès aux données via des interfaces séparées des implémentations MySQL). Cette architecture en couches, inspirée des principes de la POO, facilite la maintenabilité et la testabilité du code.

La sécurité est au cœur du projet : authentification par JWT stocké en cookie HttpOnly (protection XSS), hachage des mots de passe par bcrypt (coût 12), validation d'email par token hashé, rate-limiting sur les routes sensibles, configuration CORS stricte avec origines autorisées explicites.

### 4.2 Base de données

La base MySQL 8 comprend 17 tables avec contraintes de clés étrangères, index optimisés et triggers. Le schéma a été conçu pour supporter la recherche avancée multi-critères (recettes par ingrédient, tag, catégorie, temps de préparation). Les scripts SQL sont versionnés et permettent la réinitialisation complète de l'environnement de développement (npm run db:reset) ainsi que le chargement de données de démonstration (npm run db:reset:demo).

### 4.3 Compétences mobilisées

| Bloc 2 Développement Back-End          |                                                                                                                                                                                                                  |                                                            |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Compétence                             | Mise en œuvre dans Recipe Shelter                                                                                                                                                                                | Preuve / Livrable                                          |
| <b>Programmation Orientée Objet</b>    | Toutes les classes métier (AuthService, RecipeService, CommentsService...) utilisent les principes POO : encapsulation, injection de dépendances via constructeurs, interfaces TypeScript pour les repositories. | <i>src/services/, src/repositories/</i>                    |
| <b>Authentification &amp; sécurité</b> | JWT signé avec secret de 256 bits, stocké en cookie HttpOnly SameSite=lax. Bcrypt coût 12. Validation d'email par token SHA-256 hashé. Rate-limiting sur /auth/*.                                                | <i>auth.service.ts, session-cookie.ts, require-auth.ts</i> |
| <b>Gestion des utilisateurs</b>        | Inscription, connexion, déconnexion, validation email, oubli de mot de passe, modification de profil. Rôles : user / admin. Bannissement avec log.                                                               | <i>src/services/auth/, src/services/users/</i>             |
| <b>CRUD complet</b>                    | Opérations create/read/update/delete implémentées pour recettes, commentaires, favoris, utilisateurs et catégories, via les repositories.                                                                        | <i>src/repositories/ (toutes entités)</i>                  |
| <b>Recherche avancée</b>               | Requêtes SQL dynamiques paramétrées pour la recherche multi-critères : mots-clés, catégorie, tags, ingrédients, temps max de préparation, pagination.                                                            | <i>recipe.repository.mysql.ts — searchPublished()</i>      |

|                              |                                                                                                                                               |                                                      |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>Base de données MySQL</b> | 17 tables, clés étrangères, index sur colonnes fréquemment filtrées (status, publishedAt, userId). Collation utf8mb4 pour le support Unicode. | <i>database/migrations/1_create_schema.sql</i>       |
| <b>Tests unitaires</b>       | Tests des DTOs (parseXxxBody), du controller d'authentification et des règles de validation, avec le test runner natif Node.js.               | <i>tests/api/ (*.test.ts)</i>                        |
| <b>Envoi d'emails</b>        | Nodemailer pour validation email, reset password et formulaire de contact. Templates d'email en HTML.                                         | <i>src/services/auth/email-validation.service.ts</i> |

## 5. Bloc 3 — Framework Angular

### 5.1 Justification du choix d'Angular

Angular 21 a été choisi comme framework front-end pour sa richesse fonctionnelle et sa cohérence avec les exigences du bloc 3. Il fournit nativement le routage, la gestion des formulaires, les intercepteurs HTTP, la gestion d'état (Signals), l'injection de dépendances et le SSR. Ce choix permet de construire une SPA complète sans dépendances tierces supplémentaires pour les fonctionnalités de base.

L'architecture retenue utilise les composants standalone (sans NgModule), les routes lazy-loaded par domaine fonctionnel et les Signals Angular pour la réactivité. Chaque domaine (auth, recipes, admin, profile, favorites...) est autonome et isolé, ce qui facilite la maintenance.

### 5.2 Compétences mobilisées

| Bloc 3 Développement avec Framework Angular |                                                                                                                                                                       |                                               |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Compétence                                  | Mise en œuvre dans Recipe Shelter                                                                                                                                     | Preuve / Livrable                             |
| <b>Affichage dynamique</b>                  | Récupération et affichage des recettes via l'API REST du back-end. Composants récupèrent les données via des services Angular injectés, binding dans les templates.   | <i>src/app/pages/recipes/, core/services/</i> |
| <b>Gestion de l'état</b>                    | Angular Signals pour l'état réactif (utilisateur connecté, favoris, liste de recettes). Signal-based computed et effects pour les dépendances réactives.              | <i>SessionService, AuthService (signals)</i>  |
| <b>Routage</b>                              | Angular Router avec routes lazy-loaded par domaine fonctionnel. Guards authGuard, guestGuard et adminGuard protègent les sections restreintes. Résolveurs de données. | <i>app.routes.ts, *.routes.ts par domaine</i> |
| <b>Interactivité sans rechargement</b>      | Formulaires de création/édition de recettes soumis via HttpClient (AJAX). Ajout/suppression de favoris et envoi de commentaires sans rechargement de page.            | <i>RecipeFormComponent, CommentsComponent</i> |
| <b>SSR (Server-Side Rendering)</b>          | Angular SSR avec Express intégré. Les pages sont rendues côté serveur pour le premier chargement, puis hydratées côté client (SPA).                                   | <i>server.ts, angular.json SSR config</i>     |
| <b>Intercepteur HTTP</b>                    | authInterceptor ajoute le header Authorization Bearer à toutes les requêtes authentifiées. Gestion centralisée des erreurs 401.                                       | <i>core/interceptors/auth.interceptor.ts</i>  |
| <b>Tests framework</b>                      | Tests unitaires Vitest sur les composants, pipes et services. Configuration ng test intégrée.                                                                         | <i>*.spec.ts, vitest.config.ts</i>            |
| <b>Conventions &amp; qualité</b>            | ESLint angular-eslint pour TypeScript et templates. Stylelint pour CSS. Prettier pour le formatage. Husky + lint-staged pour les pre-commit hooks.                    | <i>.eslintrc, .stylelintrc, husky config</i>  |

## 6. Compétences transversales mobilisées

### 6.1 Méthodologie et organisation

- Gestion du code source avec Git : branches de fonctionnalité, commits conventionnels (feat:, fix:, docs:, refactor:), pull requests et code reviews
- Organisation du projet en quatre dépôts distincts (frontend statique, frontend, backend, documentation) avec conventions de nommage documentées
- Documentation technique vivante : README complet, documentation API HTML, collections Postman, schéma de base de données en SVG
- Utilisation de variables d'environnement pour séparer les configurations de développement et de production

### 6.2 Sécurité web

- Protection contre le Cross-Site Scripting (XSS) : cookie HttpOnly, échappement automatique par Angular
- Protection CSRF : politique SameSite=lax sur le cookie de session
- Contrôle des origines CORS : whitelist explicite des origines autorisées, pas de wildcard avec credentials
- Hashage des mots de passe avec bcrypt (coût 12), jamais stockés en clair
- Rate-limiting sur les routes d'authentification pour limiter les attaques par force brute
- Validation stricte des entrées utilisateur côté serveur (DTO parsing) en complément de la validation côté client

### 6.3 Architecture et qualité logicielle

- Séparation claire des responsabilités : présentation (Angular), logique métier (Services), accès aux données (Repositories)
- Principes SOLID appliqués : interfaces pour les repositories, injection de dépendances, responsabilité unique par classe
- Gestion centralisée des erreurs HTTP avec codes d'erreur métier explicites (AUTH\_INVALID\_CREDENTIALS, RECIPES\_NOT\_FOUND...)
- Utilisation de TypeScript strict mode pour la sécurité des types sur les deux couches (front et back)
- Pipeline qualité automatisé : ESLint, Stylelint, Prettier, tests unitaires, tous vérifiés à chaque commit via Husky

### 6.4 Déploiement

- Build de production Angular (bundle minifié, tree-shaking, compilation AOT)
- Build TypeScript backend (compilation vers dist/) pour la production
- Configuration multi-environnement : environment.ts (dev) et environment.prod.ts (prod) côté Angular
- Scripts de base de données versionnés pour reproductibilité des environnements (db:reset, db:reset:demo)

## 7. Bilan et perspectives

### 7.1 Acquis du projet

Recipe Shelter m'a permis de mettre en pratique l'ensemble du cycle de développement d'une application web moderne : conception du modèle de données, développement back-end avec architecture en couches, développement front-end avec un framework JavaScript, mise en place de la sécurité (authentification, autorisation, protection des données), et livraison d'une documentation technique complète.

La complexité du projet — 17 tables en base, un cycle de vie de recette multi-états, un système de modération, des rôles utilisateur distincts — m'a conduit à structurer le code de façon à le rendre maintenable et évolutif, en appliquant des patterns reconnus (Repository, Service Layer, DTO).

### 7.2 Points d'amélioration identifiés

- Ajout de tests d'intégration pour valider les flux complets (inscription → validation email → connexion → publication)
- Mise en place d'un pipeline CI/CD (GitHub Actions) pour automatiser les tests et le déploiement
- Implémentation d'un système de notifications en temps réel (WebSocket ou Server-Sent Events) pour les nouvelles recettes et commentaires
- Gestion des images avec génération de thumbnails côté serveur et utilisation du format WebP